```
    if (!value) return;
    setIsloading(true);
    setMessages([...conversationMessages,
{ role: 'user', content: input }]);
    const { messages, newMessage } = await continueConversation([...
conversationMessages,
{ role: 'user', content: input }], provider, model);
    let textContent = '';
    for await (const delta of readStreamableValue(newMessage)) {.
      textContent = `${textContent}${delta}`;
      setMessages([...messages,
{ role: 'assistant', content: textContent }]);
    }
    setIsloading(false);
  };

  // ... (rest of the component code)

  return (
    // ... (JSX for the chat interface)
  );
};

export default Chat;
```

Updates the local state immediately with the user's message for a responsive UI

Uses readStreamableValue to asynchronously read the streamed response from the server, updating the UI in real time as new content arrives

Calls the continueConversation server action directly, passing the updated conversation history

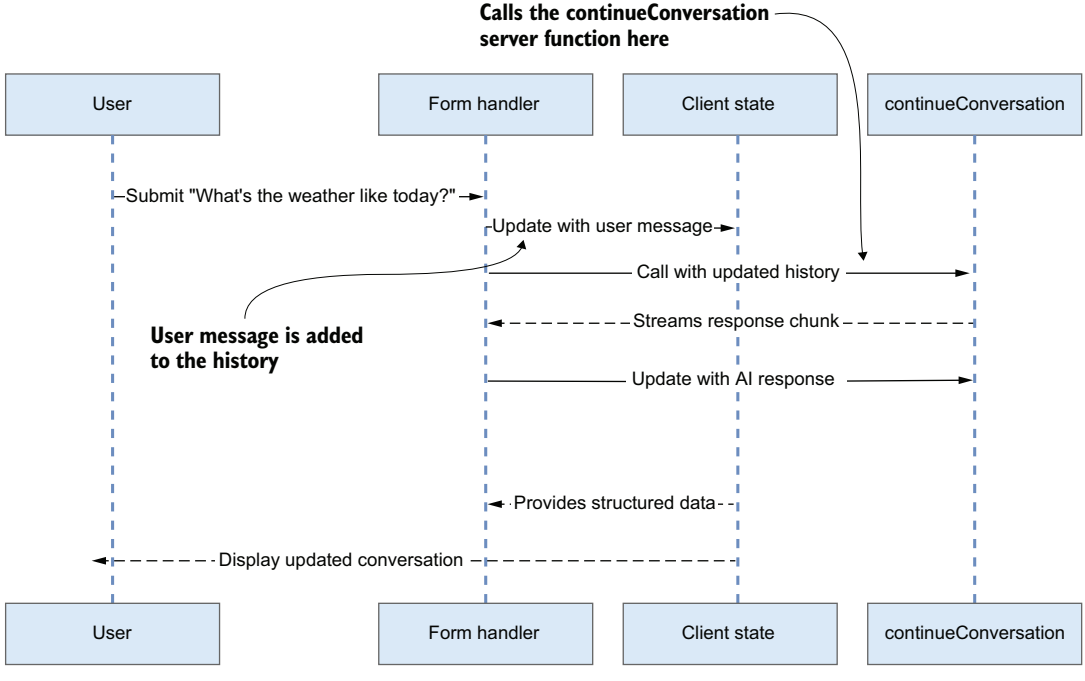


Figure 4.5 UI interaction using server actions. The user submits a message, and the form submission takes place. The call to the `continueConversation` server action happens on the server and streams the response back to the client.